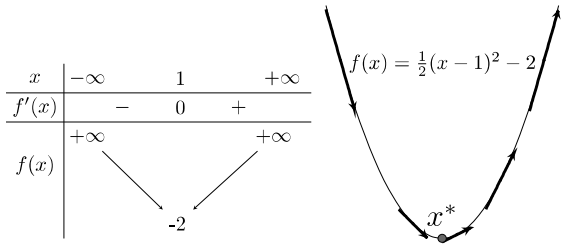


Hình 6.1. Khảo sát sự biến thiên của một đa thức bậc hai.

hàm âm, các điểm bên phải có đạo hàm dương. Với hàm số này, càng xa về phía trái của $\mathbf{x}^*$ thì đạo hàm càng âm, càng xa về phía phải thì đạo hàm càng dương.

Trong học máy nói riêng và toán tối ưu nói chung, chúng ta thường xuyên phải tìm các cực tiểu toàn cục của một hàm số. Nếu chỉ xét riêng các hàm khả vi, việc giải phương trình đạo hàm bằng không có thể phức tạp hoặc có vô số nghiệm. Thay vào đó, người ta thường tìm các điểm cực tiểu địa phương, và coi đó là một nghiệm cần tìm của bài toán trong những trường hợp nhất định.

Các điểm cực tiểu địa phương là nghiệm của phương trình đạo hàm bằng không (ta vẫn đang giả sử rằng các hàm này liên tục và khả vi). Nếu tìm được toàn bộ (hữu hạn) các điểm cực tiểu địa phương, ta chỉ cần thay từng điểm đó vào hàm số để suy ra điểm cực tiểu toàn cục. Tuy nhiên, trong hầu hết các trường hợp, việc giải phương trình đạo hàm bằng không là bất khả thi. Nguyên nhân có thể đến từ sự phức tạp của đạo hàm, từ việc các điểm dữ liệu có số chiều lớn hoặc từ việc có quá nhiều điểm dữ liệu. Thực tế cho thấy, trong nhiều bài toán machine learning, các điểm cực tiểu địa phương thường cho kết quả tốt, đặc biệt là trong các mạng neuron nhân tạo.

Một hướng tiếp cận phổ biến để giải quyết các bài toán tối ưu là dùng một phép toán. Đầu tiên, chọn một *điểm xuất phát* rồi tiến dần đến *đích* sau mỗi vòng lặp. Hạ Gradient - Gradient descent (GD) và các biến thể của nó là một trong những phương pháp được dùng nhiều nhất.

Chú ý: Khái niệm *nghiệm* của một bài toán tối ưu được sử dụng không hẳn để chỉ cực tiểu toàn cục. Nó được sử dụng theo nghĩa là kết quả của quá trình tối ưu. Kết quả ở một vòng lặp trung gian được gọi là *vị trí của nghiệm*. Nói cách khác, *nghiệm* có thể được hiểu là giá trị hiện tại của tham số cần tìm trong quá trình tối ưu.

6.2. Hạ gradient cho hàm một biến

Xét các hàm số một biến $f : \mathbb{R} \to \mathbb{R}$. Quay trở lại Hình 6.1 và một vài quan sát đã nêu. Giả sử x_t là điểm tìm được sau vòng lặp thứ t. Ta cần tìm một thuật toán để đưa x_t về càng gần x^* càng tốt. Có hai quan sát sau đây:

- Nếu đạo hàm của hàm số tại x_t là dương ($f'(x_t) > 0$) thì x_t nằm về bên phải so với x^*, và ngược lại. Để điểm tiếp theo x_{t+1} gần với x^* hơn, ta cần di chuyển x_t về bên trái, tức về phía *âm*. Nói các khác, *ta cần di chuyển x_t ngược dấu với đạo hàm*:
$$x_{t+1} = x_t + \Delta. \tag{6.1}$$
Trong đó Δ là một đại lượng ngược dấu với đạo hàm $f'(x_t)$.

- x_t càng xa x^* về bên phải thì $f'(x_t)$ càng lớn (và ngược lại). Một cách tự nhiên nhất, ta chọn lượng di chuyển Δ tỉ lệ thuận với $-f'(x_t)$.

Từ hai nhận xét trên, ta có công thức cập nhật đơn giản là

$$\boxed{x_{t+1} = x_t - \eta f'(x_t)} \tag{6.2}$$

Trong đó η là một số dương được gọi là *tốc độ học* (*learning rate*). Dấu trừ thể hiện việc x_t cần *đi ngược* với đạo hàm $f'(x_t)$. Tên gọi *Hạ gradient* xuất phát từ đây. Mặc dù các quan sát này không đúng trong mọi trường hợp, chúng vẫn là nền tảng cho rất nhiều phương pháp tối ưu.

6.2.1. Ví dụ đơn giản với Python

Xét hàm số $f(x) = x^2 + 5\sin(x)$ với đạo hàm $f'(x) = 2x + 5\cos(x)$. Giả sử xuất phát từ một điểm x_0, quy tắc cập nhật tại vòng lặp thứ t là

$$x_{t+1} = x_t - \eta(2x_t + 5\cos(x_t)). \tag{6.3}$$

Khi thực hiện trên Python, ta cần viết các hàm số

a. `grad` để tính đạo hàm.

b. `cost` để tính giá trị của hàm số. Ta không sử dụng hàm này trong thuật toán cập nhật nghiệm. Tuy nhiên, nó vẫn đóng vai trò quan trọng trong việc kiểm tra tính chính xác của đạo hàm và sự biến thiên của hàm số sau mỗi vòng lặp.

c. `myGD1` là phần chính thực hiện thuật toán GD. Đầu vào của hàm số này là điểm xuất phát `x0` và tốc độ học `eta`. Đầu ra là nghiệm của bài toán. Thuật toán dừng lại khi đạo hàm đủ nhỏ.

```
def grad(x):
    return 2*x+ 5*np.cos(x)

def cost(x):
    return x**2 + 5*np.sin(x)
```

```
def myGD1(x0, eta):
    x = [x0]
    for it in range(100):
        x_new = x[-1] - eta*grad(x[-1])
        if abs(grad(x_new)) < 1e-3: # just a small number
            break
        x.append(x_new)
    return (x, it)
```

Điểm xuất phát khác nhau

Sau khi đã có các hàm cần thiết, chúng ta thử tìm nghiệm với các điểm xuất phát khác nhau là $x_0 = -5$ và $x_0 = 5$ với cùng tốc độ học $\eta = 0.1$.

```
(x1, it1) = myGD1(-5, .1)
(x2, it2) = myGD1(5, .1)
print('Solution x1 = %f, cost = %f, after %d iterations'\
      %(x1[-1], cost(x1[-1]), it1))
print('Solution x2 = %f, cost = %f, after %d iterations'\
      %(x2[-1], cost(x2[-1]), it2))
```

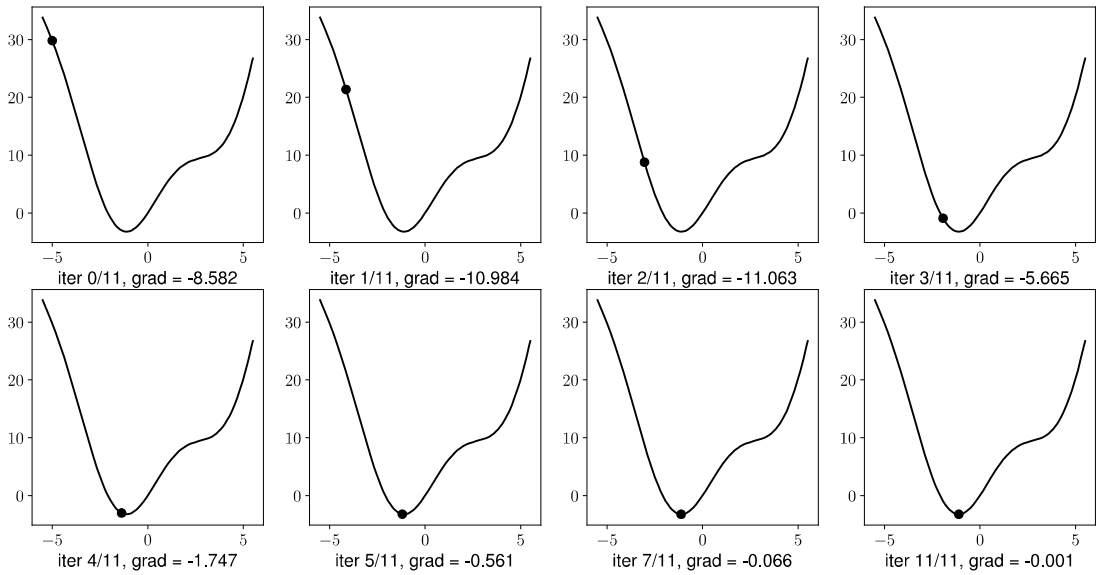
Kết quả:

```
Solution x1 = -1.110667, cost = -3.246394, after 11 iterations
Solution x2 = -1.110341, cost = -3.246394, after 29 iterations
```

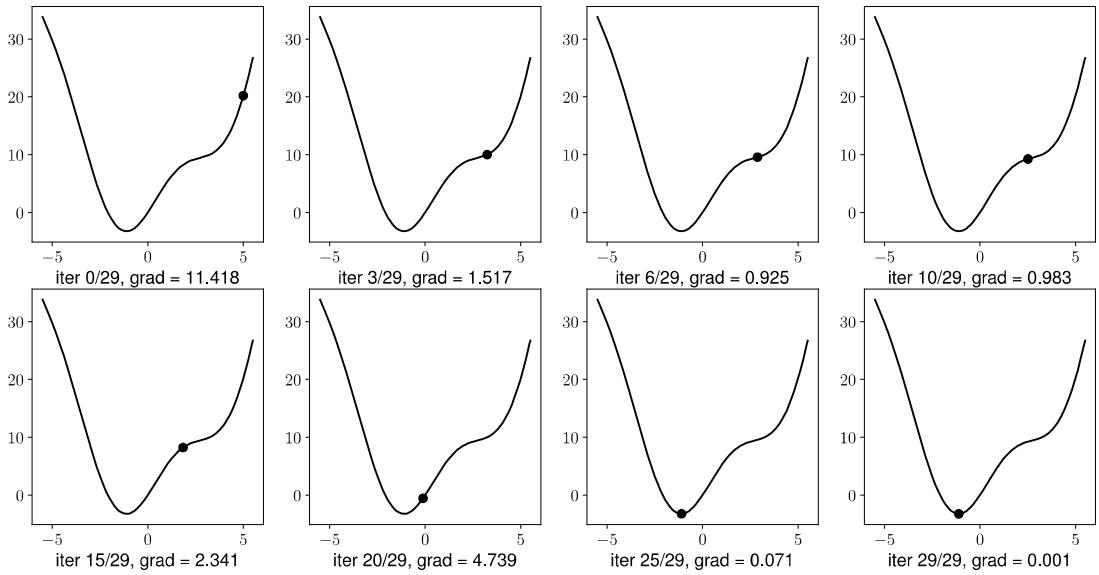
Như vậy, thuật toán trả về kết quả gần giống nhau với các điểm xuất phát khác nhau, nhưng tốc độ hội tụ khác nhau. Hình 6.2 và Hình 6.3 thể hiện vị trí của x_t và đạo hàm qua các vòng lặp với cùng tốc độ học $\eta = 0.1$ nhưng điểm xuất phát khác nhau tại -5 và 5.

Hình 6.2 tương ứng với $x_0 = -5$, thuật toán hội tụ nhanh hơn. Hơn nữa, đường đi tới đích khá suôn sẻ với đạo hàm luôn âm và trị tuyệt đối của đạo hàm nhỏ dần khi x_t tiến gần tới đích.

Hình 6.3 tương ứng với $x_0 = 5$, đường đi của x_t chứa một khu vực có đạo hàm khá nhỏ gần điểm có hoành độ bằng 2.5. Điều này khiến thuật toán la cà ở đây khá lâu. Khi vượt qua được điểm này thì mọi việc diễn ra tốt đẹp. Các điểm không phải là điểm cực tiểu nhưng có đạo hàm gần bằng không rất dễ gây ra hiện tượng x_t bị *bẫy* vì đạo hàm nhỏ khiến nó không thay đổi nhiều ở vòng lặp tiếp theo. Chúng ta sẽ thấy một kỹ thuật khác giúp thuật toán *thoát* những chiếc bẫy này.



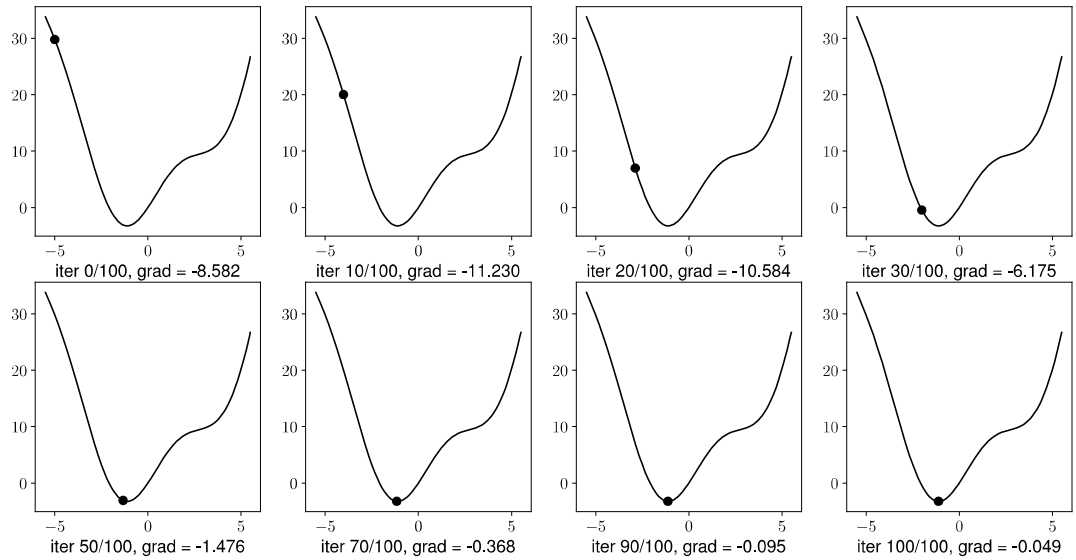
Hình 6.2. Kết quả tìm được qua các vòng lặp với $x_0 = -5, \eta = 0.1$



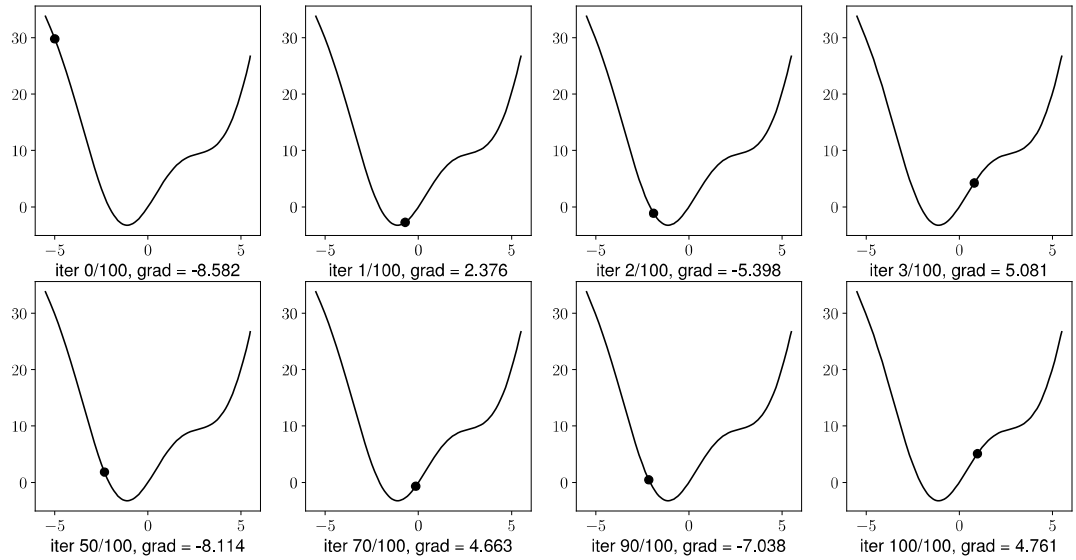
Hình 6.3. Kết quả tìm được qua các vòng lặp với $x_0 = 5, \eta = 0.1$

Tốc độ học khác nhau

Tốc độ hội tụ của GD không những phụ thuộc vào điểm xuất phát mà còn phụ thuộc vào tốc độ học. Hình 6.4 và Hình 6.5 thể hiện vị trí của x_t qua các vòng lặp với cùng điểm xuất phát $x_0 = -5$ nhưng tốc độ học khác nhau. Ta quan sát thấy hai điều:



Hình 6.4. Kết quả tìm được qua các vòng lặp với $x_0 = -5$, $\eta = 0.01$.



Hình 6.5. Kết quả tìm được qua các vòng lặp với $x_0 = -5, \eta = 0.5$

- Với tốc độ học nhỏ $\eta = 0.01$ (Hình 6.4), tốc độ hội tụ rất chậm. Trong ví dụ này ta chọn tối đa 100 vòng lặp nên thuật toán dừng lại trước khi tới đích, mặc dù đã rất gần. Trong thực tế, khi việc tính toán trở nên phức tạp, tốc độ học quá thấp sẽ ảnh hưởng nhiều tới tốc độ của thuật toán. Thậm chí x_t có thể không bao giờ tới được đích.

- Với tốc độ học lớn $\eta = 0.5$ (Hình 6.5), x_t tiến nhanh tới gần đích sau vài vòng lặp. Tuy nhiên, thuật toán không hội tụ được vì sự thay đổi vị trí của x_t sau

mỗi vòng lặp là quá lớn, khiến x_t dao động quanh đích nhưng không tới được đích.

Việc lựa chọn tốc độ học rất quan trọng. Tốc độ học thường được chọn thông qua các thí nghiệm. Ngoài ra, GD có thể làm việc hiệu quả hơn bằng cách chọn tốc độ học khác nhau ở mỗi vòng lặp. Trên thực tế, một kỹ thuật thường được sử dụng có tên là *suy giảm tốc độ học* (*learning rate decay*). Trong kỹ thuật này, tốc độ học được giảm đi sau một vài vòng lặp để nghiệm không bị dao động mạnh khi gần đích hơn.

6.3. Hạ gradient cho hàm nhiều biến

Giả sử ta cần tìm cực tiểu toàn cục cho hàm $f(\theta)$ trong đó θ là tập hợp các tham số cần tối ưu. Gradient của hàm số đó tại một điểm θ bất kỳ được ký hiệu là $\nabla_\theta f(\theta)$. Tương tự như hàm một biến, thuật toán GD cho hàm nhiều biến cũng bắt đầu bằng một điểm dự đoán θ_0, sau đó sử dụng quy tắc cập nhật

$$\boxed{\theta_{t+1} = \theta_t - \eta \nabla_\theta f(\theta_t)} \tag{6.4}$$

Hoặc viết dưới dạng đơn giản hơn: $\theta \leftarrow \theta - \eta \nabla_\theta f(\theta)$.

Quay lại với bài toán hồi quy tuyến tính

Trong mục này, chúng ta quay lại với bài toán hồi quy tuyến tính và thử tối ưu hàm mất mát của nó bằng thuật toán GD.

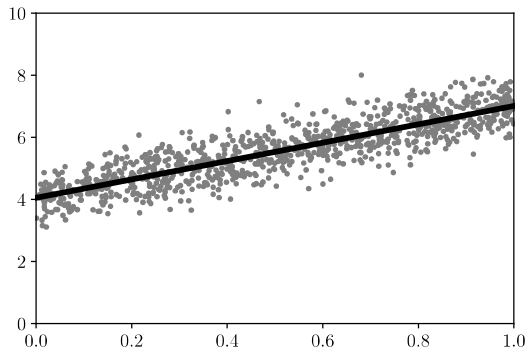
Nhắc lại hàm mất mát của hồi quy tuyến tính và gradient theo $\mathbf{w}$:

$$\mathcal{L}(\mathbf{w}) = \frac{1}{2N}\|\mathbf{y} - \mathbf{X}^T\mathbf{w}\|_2^2; \quad \nabla_\mathbf{w}\mathcal{L}(\mathbf{w}) = \frac{1}{N}\mathbf{X}(\mathbf{X}^T\mathbf{w} - \mathbf{y}) \tag{6.5}$$

Ví dụ trên Python và một vài lưu ý khi lập trình

Trước tiên, chúng ta tạo 1000 điểm dữ liệu gần đường thẳng $y = 4 + 3x$ rồi dùng thư viện scikit-learn để tìm nghiệm cho hồi quy tuyến tính:

```
from sklearn.linear_model import LinearRegression
X = np.random.rand(1000)
y = 4 + 3 * X + .5*np.random.randn(1000) # noise added
model = LinearRegression()
model.fit(X.reshape(-1, 1), y.reshape(-1, 1))
w, b = model.coef_[0][0], model.intercept_[0]
sol_sklearn = np.array([b, w])
print(sol_sklearn)
```



Hình 6.6. Nghiệm của bài toán hồi quy tuyến tính (đường thẳng màu đen) tìm được bằng thư viện scikit-learn.

Kết quả:

```
Solution found by sklearn: [ 3.94323245  3.12067542]
```

Các điểm dữ liệu và đường thẳng tìm được bằng hồi quy tuyến tính có phương trình $y \approx 3.94 + 3.12x$ được minh hoạ trong Hình 6.6. Nghiệm tìm này được rất gần với mong đợi.

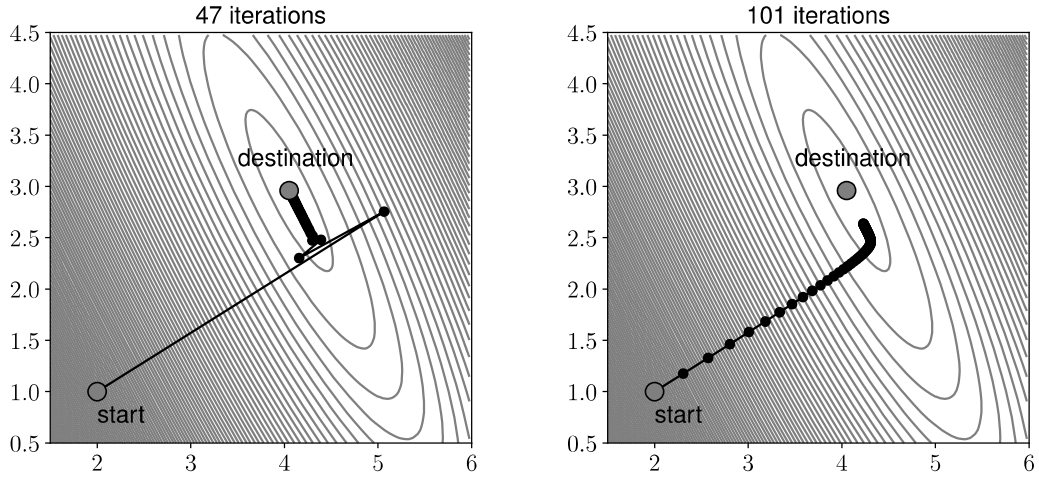
Tiếp theo, ta sẽ thực hiện tìm nghiệm bằng GD. Ta cần viết hàm mất mát và gradient theo $\mathbf{w}$. Chú ý rằng ở đây $\mathbf{w}$ đã bao gồm hệ số điều chỉnh b.

```
def grad(w):
    N = Xbar.shape[0]
    return 1/N * Xbar.T.dot(Xbar.dot(w) - y)

def cost(w):
    N = Xbar.shape[0]
    return .5/N*np.linalg.norm(y - Xbar.dot(w))**2
```

```
def myGD(w_init, grad, eta):
    w = [w_init]
    for it in range(100):
        w_new = w[-1] - eta*grad(w[-1])
        if np.linalg.norm(grad(w_new))/len(w_new) < 1e-3:
            break
        w.append(w_new)
    return w, it

one = np.ones((X.shape[0],1))
Xbar = np.concatenate((one, X.reshape(-1, 1)), axis = 1)
w_init = np.array([[2], [1]])
w1, it1 = myGD(w_init, grad, 1)
print('Sol found by GD: w = ', w1[-1].T, ', after %d iterations.' %(it1+1))
```



(a) $\eta = 1$. **(b)** $\eta = 0.1$.

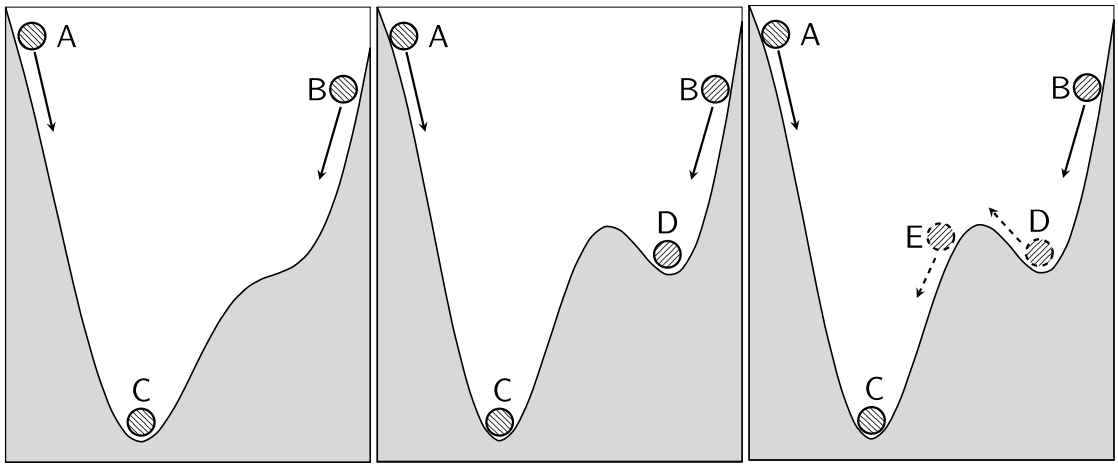
Hình 6.7. Đường đi nghiệm của hồi quy tuyến tính với các tốc độ học khác nhau.

Kết quả:

```
Sol found by GD: w =  [ 3.99026984  2.98702942] , after 49 iterations.
```

Thuật toán hội tụ tới kết quả khá gần với nghiệm tìm được theo scikit-learn sau 49 vòng lặp. Hình 6.7 mô tả đường đi của $\mathbf{w}$ với cùng điểm xuất phát nhưng tốc độ học khác nhau. Các điểm được đánh dấu 'start' là các điểm xuất phát. Các điểm được đánh dấu 'destination' là nghiệm tìm được bằng thư viện scikit-learn. Các điểm hình tròn nhỏ màu đen là vị trí của $\mathbf{w}$ qua các vòng lặp trung gian. Ta thấy rằng khi $\eta = 1$, thuật toán hội tụ tới rất gần đích theo thư viện sau 49 vòng lặp. Với tốc độ học nhỏ hơn, $\eta = 0.1$, nghiệm vẫn còn cách xa đích sau hơn 100 vòng lặp. Như vậy, việc chọn tốc độ học hợp lý là rất quan trọng.

Ở đây, chúng ta cùng làm quen với một khái niệm quan trọng: *đường đồng mức*. Khái niệm này thường xuất hiện trong các bản đồ tự nhiên. Với các ngọn núi, đường đồng mức là các đường kín bao quanh đỉnh núi, bao gồm các điểm có cùng độ cao so với mực nước biển. Khái niệm tương tự cũng được sử dụng trong tối ưu. Đường đồng mức của một hàm số là tập hợp các điểm làm cho hàm số có cùng giá trị. Xét một hàm số hai biến với đồ thị là một bề mặt trong không gian ba chiều. Các đường đồng mức là giao điểm của bề mặt này với các mặt phẳng song song với đáy. Hàm mất mát của hồi quy tuyến tính với dữ liệu một chiều là một hàm bậc hai theo hai thành phần trong vector trọng số $\mathbf{w}$. Đồ thị của nó là một bề mặt parabolic. Vì vậy, các đường đồng mức của hàm này là các đường ellipse có cùng tâm như trên Hình 6.7. Tâm này chính là đáy của parabolic và là giá trị nhỏ nhất của hàm mất mát. Các đường đồng mức càng gần tâm ('destination') tương ứng với giá trị càng thấp.



(a) GD (b) GD (c) GD với momentum

Hình 6.8. So sánh GD với các hiện tượng vật lý.

6.4. Hạ gradient với momentum

Trước hết, nhắc lại thuật toán GD để tối ưu một hàm mất mát $J(\theta)$:

- Dự đoán một điểm xuất phát $\theta = \theta_0$.
- Cập nhật θ theo công thức

$$\theta \leftarrow \theta - \eta \nabla_\theta J(\theta) \tag{6.6}$$

  tới khi hội tụ. Ở đây, $\nabla_\theta J(\theta)$ là gradient của hàm mất mát tại θ.

Gradient dưới góc nhìn vật lý

Thuật toán GD thường được ví với tác dụng của trọng lực lên một hòn bi đặt trên một mặt có dạng thung lũng như Hình 6.8a. Bất kể ta đặt hòn bi ở A hay B thì cuối cùng nó cũng sẽ lăn xuống và kết thúc ở vị trí C.

Tuy nhiên, nếu bề mặt có hai đáy thung lũng như Hình 6.8b thì tùy vào việc đặt bi ở A hoặc B, vị trí cuối cùng tương ứng của bi sẽ ở C hoặc D (giả sử rằng ma sát đủ lớn và đà không mạnh để bi có thể vượt dốc). Điểm D là một điểm cực tiểu địa phương, điểm C là điểm cực tiểu toàn cục.

Vẫn trong Hình 6.8b, nếu vận tốc ban đầu của bi ở điểm B đủ lớn, nó vẫn có thể tiến tới dốc bên trái của D do có *đà*. Nếu vận tốc ban đầu lớn hơn nữa, bi có thể vượt dốc tới điểm E rồi lăn xuống C như trong Hình 6.8c. Dựa trên quan sát này, một thuật toán được ra đời nhằm giúp GD thoát được các cực tiểu địa phương. Thuật toán đó có tên là *momentum* (tức *theo đà*).

Hạ gradient với momentum

Làm thế nào để biểu diễn *momentum* dưới dạng toán học?

Trong GD, ta cần tính lượng thay đổi ở thời điểm t để cập nhật vị trí mới cho nghiệm (tức *hòn bi*). Nếu ta coi đại lượng này như vận tốc v_t trong vật lý, vị trí mới của *hòn bi* sẽ là $\theta_{t+1} = \theta_t - v_t$ với giả sử rằng mỗi vòng lặp là một đơn vị thời gian. Dấu trừ thể hiện việc phải di chuyển ngược với gradient. Việc tiếp theo là tính đại lượng v_t sao cho nó vừa mang thông tin của *độ dốc* hiện tại (tức gradient), vừa mang thông tin của *đà*. Thông tin của đà có thể được hiểu là vận tốc trước đó v_{t-1} (với giả sử rằng vận tốc ban đầu $v_0 = 0$). Một cách đơn giản nhất, ta có thể lấy tổng trọng số của chúng:

$$\boxed{v_t = \gamma v_{t-1} + \eta \nabla_\theta J(\theta)} \tag{6.7}$$

Trong đó γ là một số dương nhỏ hơn một. Giá trị thường được chọn là khoảng 0.9, v_{t-1} là vận tốc tại thời điểm trước đó, $\nabla_\theta J(\theta)$ chính là độ dốc tại điểm hiện tại. Từ đó, ta có công thức cập nhật nghiệm:

$$\boxed{\theta \leftarrow \theta - v_t = \theta - \eta \nabla_\theta J(\theta) - \gamma v_{t-1}} \tag{6.8}$$

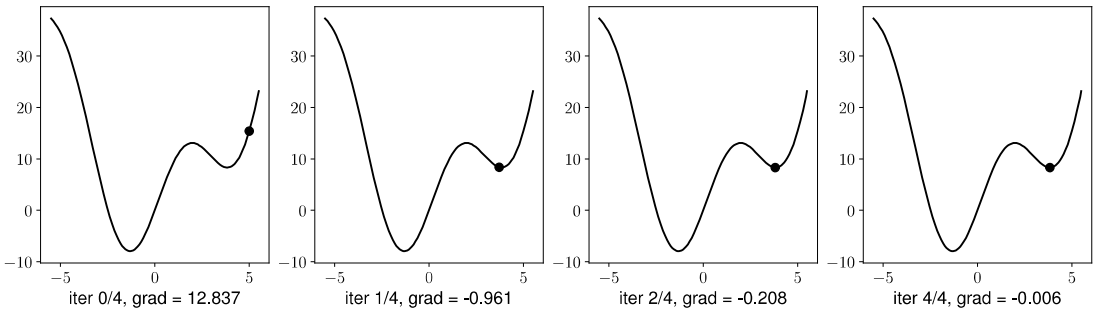
Sự khác nhau giữa GD thông thường và GD với momentem nằm ở thành phần cuối cùng trong (6.8). Thuật toán đơn giản này mang lại hiệu quả trong các bài toán thực tế.

Xét một hàm đơn giản có hai điểm cực tiểu địa phương, trong đó một điểm là cực tiểu toàn cục:

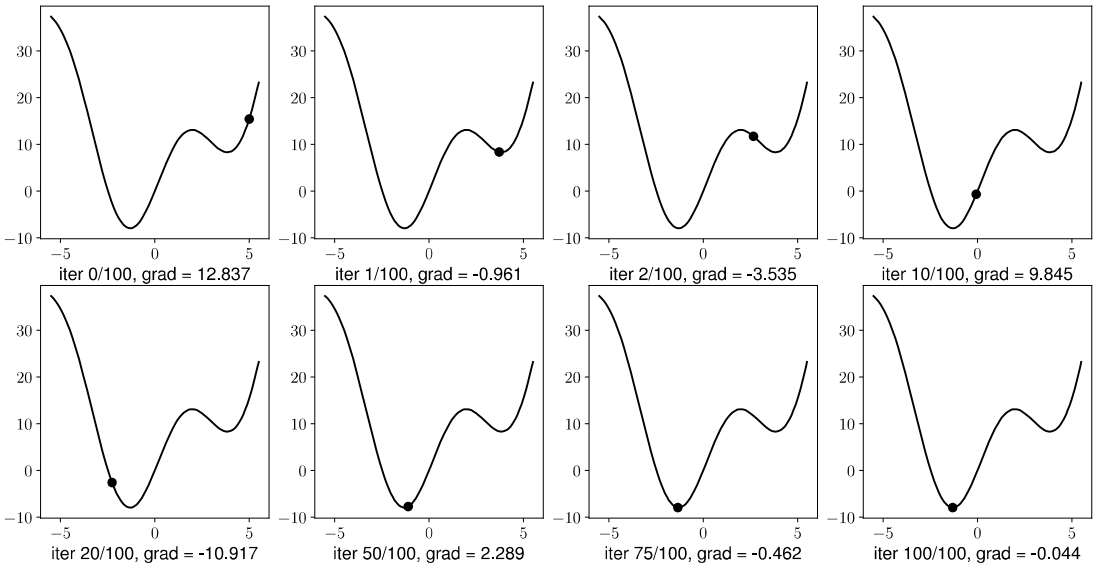
$$f(x) = x^2 + 10\sin(x). \tag{6.9}$$

Hàm số này có đạo hàm là $f'(x) = 2x + 10\cos(x)$. Hình 6.9 thể hiện các vị trí trung gian của nghiệm khi không sử dụng momentum. Ta thấy rằng thuật toán hội tụ nhanh chóng sau chỉ bốn vòng lặp. Tuy nhiên, nghiệm đạt được không phải là cực tiểu toàn cục. Trong khi đó, Hình 6.10 thể hiện các vị trí trung gian của nghiệm khi có sử dụng momentum. Chúng ta thấy rằng *hòn bi* vượt được dốc thứ nhất nhờ có *đà*, theo quán tính tiếp tục vượt qua điểm cực tiểu toàn cục, nhưng trở lại điểm này sau 50 vòng lặp rồi chuyển động chậm dần quanh đó tới khi dừng hẳn ở vòng lặp thứ 100. Ví dụ này cho thấy momentum thực sự đã giúp nghiệm thoát được khu vực cực tiểu địa phương.

Nếu biết trước điểm xuất phát `theta`, gradient của hàm mất mát tại một điểm bất kỳ `grad(theta)`, lượng thông tin lưu trữ từ vận tốc trước đó `gamma` và tốc độ học `eta`, chúng ta có thể viết hàm `GD_momentum` như sau:

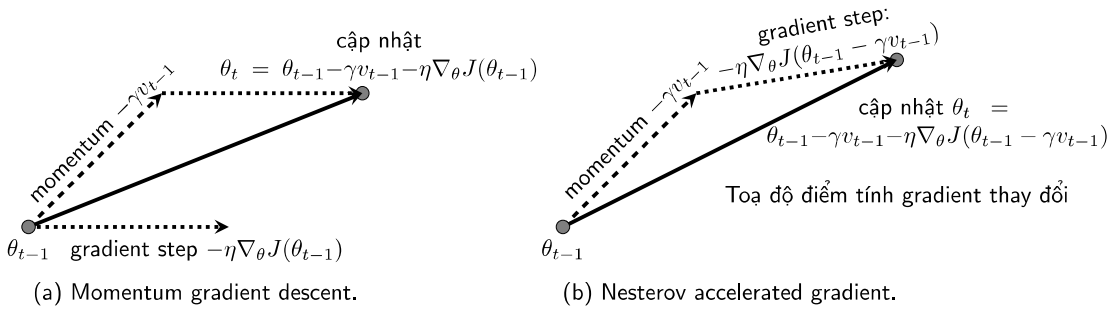


Hình 6.9. GD thông thường



Hình 6.10. GD với momentum

```
def GD_momentum(grad, theta_init, eta, gamma):
    # Suppose we want to store history of theta
    theta = [theta_init]
    v_old = np.zeros_like(theta_init)
    for it in range(100):
        v_new = gamma*v_old + eta*grad(theta[-1])
        theta_new = theta[-1] - v_new
        if np.linalg.norm(grad(theta_new))/np.array(theta_init).size < 1e-3:
            break
        theta.append(theta_new)
        v_old = v_new
    return theta
```



(a) Momentum gradient descent. (b) Nesterov accelerated gradient.

Hình 6.11. Ý tưởng của Nesterov accelerated gradient

6.5. Nesterov accelerated gradient

Momentum giúp nghiệm vượt qua được khu vực cực tiểu địa phương. Tuy nhiên, có một hạn chế có thể thấy trong ví dụ trên. Khi tới gần đích, momemtum khiến nghiệm dao động một khoảng thời gian nữa trước khi hội tụ. Một kỹ thuật có tên *Nesterov accelerated gradient* giúp cho thuật toán momentum GD hội tụ nhanh hơn.

Ý tưởng trung tâm của thuật toán là dự đoán vị trí của nghiệm trước một bước. Cụ thể, nếu sử dụng số hạng momentum γv_{t-1} để cập nhật thì vị trí tiếp theo của nghiệm là $\theta - \gamma v_{t-1}$. Vậy, thay vì sử dụng gradient tại điểm hiện tại, NAG sử dụng gradient tại điểm tiếp theo *nếu sử dụng momentum.* Ý tưởng này được thể hiện trên Hình 6.11.

6.5.1. Công thức cập nhật

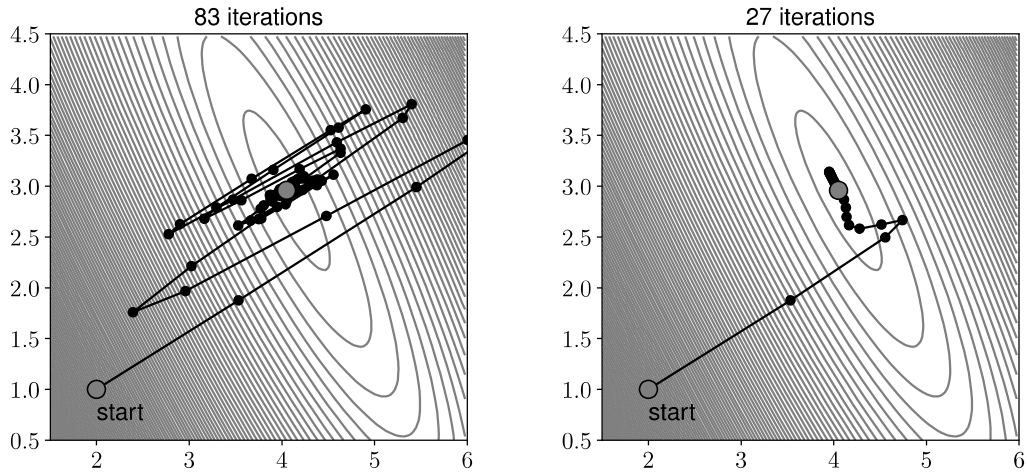
Công thức cập nhật của NAG được cho như sau:

$$v_t = \gamma v_{t-1} + \eta \nabla_\theta J(\theta - \gamma v_{t-1}) \tag{6.10}$$

$$\theta \leftarrow \theta - v_t \tag{6.11}$$

Đoạn code dưới đây thể hiện cách cập nhật nghiệm bằng NAG:

```
def GD_NAG(grad, theta_init, eta, gamma):
    theta = [theta_init]
    v = [np.zeros_like(theta_init)]
    for it in range(100):
        v_new = gamma*v[-1] + eta*grad(theta[-1] - gamma*v[-1])
        theta_new = theta[-1] - v_new
        if np.linalg.norm(grad(theta_new))/np.array(theta_init).size < 1e-3:
            break
        theta.append(theta_new)
        v.append(v_new)
return theta
```



(a) GD với momentum. **(b)** GD với NAG.

Hình 6.12. Đường đi của nghiệm cho bài toán hồi quy tuyến tính với hai phương pháp Hạ gradient khác nhau. NAG cho nghiệm mượt hơn và nhanh hơn.

6.5.2. Ví dụ minh họa

Chúng ta cùng áp dụng cả GD với momentum và GD với NAG cho bài toán hồi quy tuyến tính. Hình 6.12a thể hiện đường đi của nghiệm với phương pháp momentum. Nghiệm đi khá *zigzag* và mất nhiều vòng lặp hơn. Hình 6.12b thể hiện đường đi của nghiệm với phương pháp NAG, nghiệm hội tụ nhanh hơn và đường đi ít *zigzag* hơn.

6.6. Hạ gradient ngẫu nhiên

6.6.1. Hạ gradient theo Batch

Thuật toán GD được đề cập từ đầu bài còn được gọi là *batch gradient desenct*. Batch ở đây được hiểu là *tất cả*, tức sử dụng tất cả các điểm dữ liệu $\mathbf{x}_i$ để cập nhật bộ tham số θ. Hạn chế của việc này là khi lượng cơ sở dữ liệu lớn, việc tính toán gradient trên toàn bộ dữ liệu tại mỗi vòng lặp tốn nhiều thời gian.

Online learning là khi cơ sở dữ liệu được cập nhật liên tục, mỗi lần tăng thêm vài điểm dữ liệu mới. Việc này yêu cầu mô hình cũng phải được thay đổi để phù hợp với dữ liệu mới. Nếu thực hiện batch GD, tức tính lại gradient của hàm mất mát với toàn bộ dữ liệu, độ phức tạp tính toán sẽ rất cao. Lúc đó, thuật toán có thể không còn mang tính *online* nữa do mất quá nhiều thời gian tính toán.

Một kỹ thuật đơn giản hơn được sử dụng là *Hạ gradient ngẫu nhiên* (SGD). Thuật toán này có thể gây ra sai số nhưng mang lại lợi ích về mặt tính toán.

6.6.2. Hạ gradient ngẫu nhiên

Trong SGD, tại một thời điểm, ta tính gradient của hàm mất mát dựa trên *chỉ một* điểm dữ liệu $\mathbf{x}_i$ rồi cập nhật θ. Chú ý rằng hàm mất mát thường được lấy trung bình trên tất điểm dữ liệu nên gradient tương ứng với một điểm được kỳ vọng là khá gần với gradient tính theo mọi điểm dữ liệu. Sau khi duyệt qua tất cả các điểm dữ liệu, thuật toán lặp lại quá trình trên. Biến thể đơn giản này trên thực tế làm việc rất hiệu quả.

epoch

Mỗi lần duyệt một lượt qua tất cả các điểm trên toàn bộ dữ liệu được gọi là một *epoch*. Với GD thông thường, mỗi epoch ứng với một lần cập nhật θ. Với SGD, mỗi epoch ứng với N lần cập nhật θ với N là số điểm dữ liệu. Một mặt, việc cập nhật θ theo từng điểm có thể làm giảm tốc độ thực hiện một epoch. Nhưng mặt khác, với SGD, nghiệm có thể hội tụ sau vài epoch. Vì vậy, SGD phù hợp với các bài toán có lượng cơ sở dữ liệu lớn và các bài toán yêu cầu mô hình thay đổi liên tục như *học trực tuyến*. Với một mô hình đã được huấn luyện từ trước, khi có thêm dữ liệu, ta có thể chạy thêm một vài epoch nữa là đã có nghiệm hội tụ.

Mỗi lần cập nhật nghiệm là một vòng lặp. Mỗi lần duyệt hết toàn bộ dữ liệu là một epoch. Một epoch bao gồm nhiều vòng lặp.

Thứ tự lựa chọn điểm dữ liệu

Một điểm cần lưu ý là sau mỗi epoch, thứ tự lấy các dữ liệu cần được xáo trộn để đảm bảo tính ngẫu nhiên. Việc này cũng ảnh hưởng tới hiệu năng của SGD. Đây cũng chính là lý do thuật toán này có chứa từ *stochastic-ngẫu nhiên*.

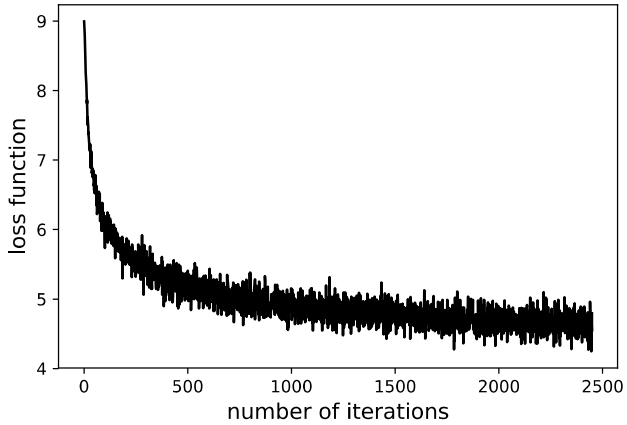
Quy tắc cập nhật của SGD là

$$\theta \leftarrow \theta - \eta \nabla_\theta J(\theta; \mathbf{x}_i, \mathbf{y}_i) \tag{6.12}$$

Trong đó $J(\theta; \mathbf{x}_i, \mathbf{y}_i) \triangleq J_i(\theta)$ là hàm mất mát nếu chỉ có một cặp dữ liệu thứ i. Các kỹ thuật biến thể của GD như momentum hay NAG hoàn toàn có thể được áp dụng vào SGD.

6.6.3. Hạ gradient theo Mini-batch

Khác với SGD, mini-batch GD sử dụng $1 < k < N$ điểm dữ liệu để cập nhật ở mỗi vòng lặp. Giống với SGD, mini-batch GD bắt đầu mỗi epoch bằng việc xáo trộn ngẫu nhiên dữ liệu rồi chia toàn bộ dữ liệu thành các mini-batch, mỗi



Hình 6.13. Ví dụ về giá trị hàm mất mát sau mỗi vòng lặp khi sử dụng mini-batch Hạ gradient. Hàm mất mát dao động sau mỗi lần cập nhật nhưng nhìn chung giảm dần và có xu hướng hội tụ.

mini-batch có k điểm dữ liệu (trừ mini-batch cuối có thể có ít hơn nếu N không chia hết cho k). Ở mỗi vòng lặp, một mini-batch được lấy ra để tính toán gradient rồi cập nhật θ. Khi thuật toán chạy hết dữ liệu một lượt cũng là khi kết thúc một epoch. Như vậy, một epoch bao gồm xấp xỉ N/k vòng lặp. Giá trị k được gọi là *kích thước batch* (không phải *kích thước mini-batch*) được chọn trong khoảng khoảng từ vài chục đến vài trăm.

Hình 6.13 là ví dụ về giá trị của hàm mất mát của một mô hình phức tạp hơn khi sử dụng mini-batch GD. Mặc dù giá trị của hàm mất mát sau các vòng lặp không luôn luôn giảm, nhìn chung giá trị này có xu hướng giảm và hội tụ.

6.7. Thảo luận

6.7.1. Điều kiện dừng thuật toán

Khi nào thì nên dừng thuật toán GD?

Trong thực nghiệm, chúng ta có thể kết hợp các phương pháp sau:

a. Giới hạn số vòng lặp. Nhược điểm của cách làm này là thuật toán có thể dừng lại trước khi nghiệm đủ tốt. Tuy nhiên, đây là phương pháp phổ biến nhất và cũng đảm bảo được chương trình chạy không quá lâu.

b. So sánh gradient của hàm mất mát tại hai lần cập nhật liên tiếp, khi nào giá trị này đủ nhỏ thì dừng lại.

c. So sánh giá trị của hàm mất mát sau một vài epoch, khi nào sự sai khác đủ nhỏ thì dừng lại. Nhược điểm của phương pháp này là nếu hàm mất mát có dạng bằng phẳng tại một điểm không phải cực tiểu địa phương, thuật toán sẽ dừng lại trước khi đạt giá trị mong muốn.

d. Vừa chạy GD, vừa kiểm tra kết quả. Một kỹ thuật khác thường được sử dụng là cho thuật toán chạy với số lượng vòng lặp lớn. Trong quá trình chạy, chương trình thường xuyên kiểm tra chất lượng mô hình trên tập huấn luyện và tập xác thực. Đồng thời, mô hình sau một vài vòng lặp được lưu lại trong bộ nhớ. Nếu ta thấy chất lượng mô hình bắt đầu giảm trên tập xác thực thì dừng lại. Đây chính là kỹ thuật *early stoping* - Dừng sớm để giảm quá khớp.

6.8. Câu hỏi ôn tập bài 6

Câu 1. Gradient Decent là gì?

Gợi ý: Gradient Descent là một thuật toán tối ưu hóa trong học máy, được sử dụng để tìm cực trị địa phương (hoặc toàn cục) của hàm mất mát (loss function).

Câu 2. Thuật toán Gradient Decent hoạt động như thế nào

Gợi ý: Thuật toán Gradient Descent hoạt động bằng cách cập nhật các tham số của mô hình dựa trên đạo hàm riêng của hàm mất mát. Cụ thể, nó sẽ tính đạo hàm riêng của hàm mất mát theo từng tham số, sau đó điều chỉnh các tham số đó bằng cách di chuyển theo hướng ngược với đạo hàm riêng đó.

Câu 3. Gradient Decent có bao nhiêu loại?

Gợi ý: Gradient Descent có 3 loại chính: Batch Gradient Descent, Stochastic Gradient Descent và Mini-Batch Gradient Descent.

Câu 4. Batch Gradient Descent là gì?

Gợi ý: Batch Gradient Descent là loại Gradient Descent cơ bản nhất, nó tính toán đạo hàm riêng của hàm mất mát cho toàn bộ dữ liệu huấn luyện. Sau đó, nó cập nhật các tham số của mô hình dựa trên tổng đạo hàm riêng đó.

Câu 5. Stochastic Gradient Descent là gì?

Gợi ý: Stochastic Gradient Descent là loại Gradient Descent sử dụng một mẫu dữ liệu ngẫu nhiên từ tập huấn luyện để tính đạo hàm riêng của hàm mất mát.

Sau đó, nó cập nhật các tham số của mô hình dựa trên đạo hàm riêng đó.

Câu 6. Mini-Batch Gradient Descent là gì?

Gợi ý: Mini-Batch Gradient Descent là loại Gradient Descent sử dụng một lượng nhỏ các mẫu dữ liệu để tính đạo hàm riêng của hàm mất mát. Sau đó, nó cập nhật các tham số của mô hình dựa trên đạo hàm riêng đó.

Câu 7. Tại sao thuật toán Gradient Descent cần được sử dụng trong học máy?

Gợi ý: Gradient Descent là một thuật toán tối ưu hóa hiệu quả và phổ biến trong học máy, vì nó cho phép mô hình học được từ dữ liệu huấn luyện để dự đoán đầu ra cho các dữ liệu mới. Nó giúp giảm thiểu hàm mất mát của mô hình, từ đó cải thiện độ chính xác.

Bài 7

Thuật toán học perceptron

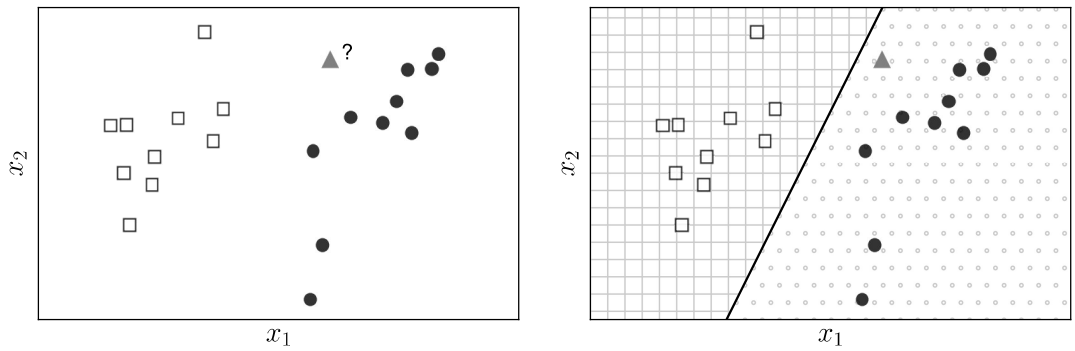
7.1. Giới thiệu

Trong bài này, chúng ta cùng tìm hiểu một trong các thuật toán xuất hiện đầu tiên trong lịch sử machine learning. Đây là một phương pháp phân loại đơn giản có tên là *thuật toán học perceptron* (perceptron learning algorithm – PLA). Thuật toán này được thiết kế cho bài toán *phân loại nhị phân* khi dữ liệu chỉ thuộc một trong hai nhãn. Đây là nền tảng cho các thuật toán liên quan tới mạng neuron nhân tạo và gần đây là deep learning.

Giả sử có hai tập dữ liệu hình vuông và tròn như được minh hoạ trong Hình 7.1a. Bài toán đặt ra là từ dữ liệu của hai tập được gán nhãn cho trước, hãy xây dựng một bộ phân loại có khả năng dự đoán nhãn của một điểm dữ liệu mới, chẳng hạn điểm hình tam giác màu xám.

Nếu coi mỗi vector đặc trưng là một điểm trong không gian nhiều chiều, bài toán phân loại có thể được coi như bài toán xác định nhãn của từng điểm trong không gian. Nếu coi mỗi nhãn chiếm một hoặc vài vùng trong không gian, ta cần đi tìm ranh giới giữa các vùng đó. Ranh giới đơn giản nhất trong không gian hai chiều là một đường thẳng, trong không gian ba chiều là một mặt phẳng, trong không gian nhiều chiều là một *siêu phẳng*. Những ranh giới phẳng này đơn giản vì chúng có thể được biểu diễn bởi một hàm số tuyến tính. Hình 7.1b minh họa một đường thẳng phân chia hai tập dữ liệu trong không gian hai chiều. Trong trường hợp này, điểm dữ liệu mới hình tam giác rơi vào cùng tập hợp với các điểm hình tròn.

PLA là một thuật toán đơn giản giúp tìm ranh giới siêu phẳng cho bài toán phân loại nhị phân trong trường hợp tồn tại siêu phẳng đó. Nếu hai tập dữ liệu có thể được phân chia hoàn toàn bằng một siêu phẳng, ta nói rằng hai tập đó *tách biệt tuyến tính* (linearly separable).



(a) **(b)**

Hình 7.1. Bài toán phân loại nhị phân trong không gian hai chiều. (a) Cho hai tập dữ liệu được gán nhãn vuông và tròn, hãy xác định nhãn của điểm tam giác. (b) Ví dụ về một ranh giới phẳng phân chia hai tập hợp. Điểm tam giác được phân vào tập các điểm hình tròn.

7.2. Thuật toán học perceptron

7.2.1. Quy tắc phân loại

Giả sử $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_N] \in \mathbb{R}^{d\times N}$ là ma trận chứa tập huấn luyện mà mỗi cột $\mathbf{x}_i$ là một điểm dữ liệu trong không gian d chiều. Các nhãn được lưu trong một vector hàng $\mathbf{y} = [y_1, y_2, \ldots, y_N] \in \mathbb{R}^{1\times N}$ với $y_i = 1$ nếu $\mathbf{x}_i$ mang nhãn vuông và $y_i = -1$ nếu $\mathbf{x}_i$ mang nhãn tròn.

Tại một thời điểm, giả sử ranh giới là một siêu phẳng có phương trình

$$f_{\mathbf{w}}(\mathbf{x}) = w_1x_1 + \cdots + w_dx_d + w_0 = \mathbf{x}^T\mathbf{w} + w_0 = 0 \tag{7.1}$$

với $\mathbf{w} \in \mathbb{R}^d$ là vector trọng số và w_0 là hệ số điều chỉnh. Bằng cách sử dụng thủ thuật gộp hệ số điều chỉnh (xem Mục 3.2.4), ta có thể coi phương trình siêu phẳng là $f_{\mathbf{w}}(\mathbf{x}) = \mathbf{x}^T\mathbf{w} = 0$ với $\mathbf{x}$ ở đây được ngầm hiểu như vector đặc trưng mở rộng thêm một đặc trưng bằng một. Vector trọng số $\mathbf{w}$ chính là *vector pháp tuyến* của siêu phẳng $\mathbf{x}^T\mathbf{w} = 0$.

Trong không gian hai chiều, giả sử đường thẳng $w_1x_1 + w_2x_2 + w_0 = 0$ là nghiệm cần tìm như Hình 7.2a. Ta thấy rằng các điểm nằm cùng phía so với đường thẳng này làm cho hàm số $f_{\mathbf{w}}(\mathbf{x})$ mang cùng dấu. Nếu cần thiết, ta có thể đổi dấu của $\mathbf{w}$ để các điểm trên nửa mặt phẳng nền kẻ ô vuông mang dấu dương (+), các điểm trên nửa mặt phẳng nền chấm mang dấu âm (-). Các dấu này tương đương với nhãn y của mỗi điểm dữ liệu. Như vậy, nếu $\mathbf{w}$ là một nghiệm của bài toán thì nhãn của một điểm dữ liệu mới $\mathbf{x}$ được xác định bởi

$$\text{label}(\mathbf{x}) = \begin{cases} 1 & \text{nếu } \mathbf{x}^T\mathbf{w} \geq 0 \\ -1 & \text{o.w.} \end{cases} \tag{7.2}$$